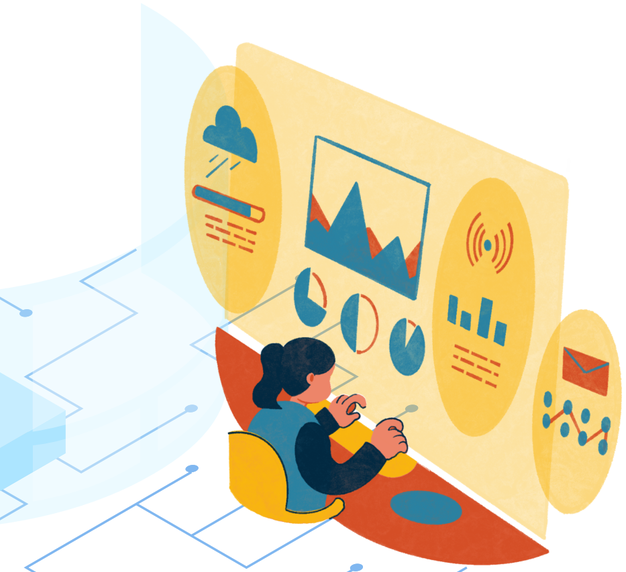


WEEK 4 · BUILDING ON CLASS & OBJECT

Encapsulation, Access Modifiers, Packages & Interfaces

Writing Java code that protects its data, stays organized, and stays flexible as it grows.

MOTIVATION

Why Do We Need Encapsulation?

A public field can be changed by anyone, from anywhere, with zero control.

```
public class Account {
  public double balance;
}

// Anywhere else in the program:
Account acc = new Account();
acc.balance = -999999;
// Java: no error at all!
```

Think About It!

Is this a bug in Java — or a bug in how the class was designed? Whose job is it to stop a negative balance from ever happening?

Encapsulation is the OOP principle that fixes exactly this: it wraps data so it can only be read or changed through code the class itself controls.

Source: Oracle Java Tutorials, "Controlling Access to Members of a Class" (docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html); GeeksforGeeks, "Encapsulation in Java" (geeksforgeeks.org/encapsulation-in-java)

CORE CONCEPT

What Is Encapsulation?

Encapsulation means bundling data (fields) and behavior (methods) into one unit, and **hiding the internal details** of an object from the outside world.

Data Hiding

Fields are marked private — no outside class can touch them directly.

Controlled Access

The only way in or out is through public methods the class itself provides.

Validation

Those methods can reject bad values before any damage is done.

Analogy: a medicine capsule keeps its ingredients sealed inside — you interact with the capsule, never the raw chemicals directly.

Source: Oracle Java Tutorials, "Controlling Access to Members of a Class" (docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html); GeeksforGeeks, "Encapsulation in Java" (geeksforgeeks.org/encapsulation-in-java)

WHY IT MATTERS

Benefits of Encapsulation

1. **Validate before you commit**
   A setter can check a new value before it ever touches the field — impossible with a public field.

2. **Hide internal representation**
   Outside code only sees what a method exposes; the object's internal complexity stays private.

3. **Change internals safely**
   You can redesign how a class stores its data without breaking any code that uses it — the public methods stay the same.

4. **Easier debugging**
   Getters and setters are natural places to set breakpoints and watch exactly when a value changes.

Try It Now!

Take the Account class from the last slide. Add a setBalance(double amount) method that only updates balance when amount >= 0, and prints “Invalid amount!” otherwise. Run it with a negative number and see the difference.

Source: GeeksforGeeks, "Encapsulation in Java" (geeksforgeeks.org/encapsulation-in-java); Oracle Java Tutorials (docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html)

Access Modifiers in Java

An access modifier controls which classes are allowed to see a field, method, or class. This is the official table from the Java Tutorials:

Modifier	Same Class	Same Package	Subclass (other pkg)	World
`public`	Y	Y	Y	Y
`protected`	Y	Y	Y	N
`no modifier (default)`	Y	Y	N	N
`private`	Y	N	N	N

Think About It!
Why does protected give subclasses in other packages a special exception, when no modifier does not? What real-world design problem might that solve?

Source: Oracle Java Tutorials, "Controlling Access to Members of a Class" (docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html)

SYNTAX

Getter & Setter Methods

```
public class Employee {
  private String name;
  private int age;

  public String getName() {
    return name;
  }

  public void setName(String name) {
    if (name != null && !name.isEmpty()) {
      this.name = name;
    } else {
      System.out.println("Name cannot be empty!");
    }
  }

  public int getAge() { return age; }
  public void setAge(int age) {
    if (age > 0) this.age = age;
    else System.out.println("Invalid age!");
  }
}
```

Getter

Returns a private field's value. Named **getFieldName()**.

Setter

Updates a private field's value. Named s**etFieldName(...)**.

This is where validation lives.

Try It Now!

- Create an Employee, then call **setAge(-5)**.
- Predict what gets printed before you run it, then run it and check.
- Now try **setAge(30)** followed by **getAge()**.

  Does the object behave the way you expected?

Source: Oracle Java Tutorials (docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html); GeeksforGeeks, "Encapsulation in Java" (geeksforgeeks.org/encapsulation-in-java)

APPLYING THE IDEA

Encapsulation in Practice: Refactoring Account

BEFORE (no encapsulation)

```
public class Account {
  public String ownerName;
  public double balance;
}

acc.balance = -999999; // allowed!
```

From the outside, using this object still looks almost the same — but the data is now impossible to corrupt.

AFTER (encapsulated)

```
public class Account {
  private String ownerName;
  private double balance;

  public Account(String name,
      double startingBalance) {
    this.ownerName = name;
    this.balance = startingBalance;
  }

  public double getBalance() {
    return balance;
  }

  public void setBalance(double amount) {
    if (amount >= 0) balance = amount;
  }
}
```

Source: Oracle Java Tutorials (docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html)

Declaring and Importing Packages

A package is a named folder for related classes — it keeps large projects from turning into one giant, name-clashing pile of files.

```
// File: mypackage/MyClass.java
package mypackage;

public class MyClass {
  // ...
}
```

```
// Option 1: import the whole package
import mypackage.*;
MyClass m = new MyClass();

// Option 2: fully qualified name
mypackage.MyClass m2 =
    new mypackage.MyClass();
```

- The package statement must be the first line of the file (before imports or the class).
- The folder structure on disk must match the package name exactly.
- Only public classes can be imported and used from another package.

Try It Now!

Create two files exactly as shown on the left. Compile MyClass.java first (javac keeps the folder structure), then write a small Main.java using Option 1 to create and use MyClass.

Source: Oracle Java Tutorials, "Creating and Using Packages" (docs.oracle.com/javase/tutorial/java/package/packages.html)

NEW TOPIC · INTERFACES

Interfaces: A New Kind of Contract

An interface lists method signatures — ***what*** a class must be able to do — without saying ***how*** it does it. It is a contract every implementing class agrees to honor.

- Contains method signatures only — no method bodies.
- Can contain constants (implicitly public, static, and final).
- A class can implement any number of interfaces — unlike extending a class, which only allows one parent.
- Two completely unrelated classes can implement the same interface.
- A class must implement every single method the interface declares.

Think About It!

A Robot and a Bird are not related by inheritance at all — but both can move(). Should Java force them to share a parent class just to prove they can both move, or is there a lighter-weight way to make that promise?

Source: Oracle Java Tutorials, "Defining an Interface" (docs.oracle.com/javase/tutorial/java/IandI/interfaceDef.html)

Defining an Interface

```
[public] interface InterfaceName {
  returnType methodOne(paramList);    // no body — ends with ;
  returnType methodTwo(paramList);

  type CONSTANT_NAME = value;  // implicitly public static final
}
```

- **No access word, or public**
  A public interface can be used by any class in any package.
- **No method bodies**
  Each method ends with a semicolon — it looks like a C-style prototype.
- **Implicitly public**
  If the interface itself is public, all its methods and constants are public automatically.
- **Implicitly static & final**
  Every variable declared in an interface is a constant — it can never change.

Source: Oracle Java Tutorials, "Defining an Interface" (docs.oracle.com/javase/tutorial/java/IandI/interfaceDef.html)

Interface Example: Animal

```
interface Animal {
  void makeSound();  // abstract method
  void move();       // abstract method
}

class Cat implements Animal {
  public void makeSound() {
    System.out.println("Meow... Meow...");
  }
  public void move() {
    System.out.println("Cat walks silently.");
  }
}

public class Main {
  public static void main(String[] args) {
    Animal a1 = new Cat();
    a1.makeSound();
    a1.move();
  }
}
```

Try It Now!

Write a Dog class that also implements Animal, with its own makeSound() and move(). Then write Animal a2 = new Dog(); and call the exact same two lines: a2.makeSound(); a2.move();

Notice: the calling code (**a1.makeSound(); a1.move();**) never changes, yet the actual behavior depends on which class is really behind the Animal reference.

That's your first real taste of ***polymorphism*** — a topic we will explore in full soon.

Source: Oracle Java Tutorials, "Defining an Interface" (docs.oracle.com/javase/tutorial/java/IandI/interfaceDef.html); example adapted from IT Del PBO course material, W04 (AY 2024/2025)

PRACTICE · TRY IT YOURSELF

Quick Exercise: Encapsulate the Element Class

The Element class below still uses public fields. Your task:

- Change every field to private
- Write a constructor with 6 parameters to set all fields
- Write a getter for every field (getName, getNumber, etc.)
- Since an Element never changes after it's created, do NOT write any setters

Try It Now!

Give yourself 5–7 minutes. Don't worry about getting it perfect — the goal is to try before we walk through the answer together.

```
class Element {
  public String name;
  public int number;
  public String symbol;
  public double mass;
  public int period;
  public int group;

  // TODO: add a constructor
  // TODO: add getters
}
```

Source: Adapted from IT Del PBO course material

Solution: The Encapsulated Element Class

```
class Element {
  private String name;
  private int number;
  private String symbol;
  private double mass;
  private int period;
  private int group;

  public Element(String name, int number, String symbol,
      double mass, int period, int group) {
    this.name = name;     this.number = number;
    this.symbol = symbol; this.mass = mass;
    this.period = period; this.group = group;
  }

  public String getName() { return name; }
  public int getNumber() { return number; }
  public String getSymbol() { return symbol; }  // ...and so on for mass, period, group
}
```

Every getter follows the exact same pattern: public <type> get<FieldName>() { return <field>; }

TIPS & HINTS

Encapsulation, Access Modifiers & Interfaces

1. **Default to private**

   Start every field as private, and only widen access when there's a clear reason to.

2. **A setter is optional**

   If a value should never change after the object is created, don't write a setter for it at all.

3. **Validate in the setter**

   The setter is the best place to guarantee an object is never left in an invalid state.

4. **An interface says WHAT, never HOW**

   It's a promise of behavior — the implementing class decides the actual logic.



COMING UP

From Interfaces to Polymorphism

- Abstract Classes — a halfway point between an interface and a full class
- Polymorphism — one interface, many behaviors, fully explored
- Dynamic Binding — how Java decides, at runtime, which version of a method to run

Today's Animal/Cat example was just a preview — keep it in mind, we'll come back to it.